Physics and Mathematics Reference

Generated 2026-03-19

1 Physical system overview

The audited execution graph models three coupled analysis layers.

1. A MATLAB heating model evaluates a discrete operating grid for a resistive nichrome coil inside a heater tube, followed by an aeration-tube network embedded in the vermicomposter bed `vermicomposter_air_heater_design_model.m:L1172-L1677`; `vermicomposter_air_heater_design_model.m:L1803-L2248`.
2. A Python summer branch replaces the electrical heater with a spot-cooled plenum and assist blower, then reuses the same reduced-order aeration, evaporation, and two-node bed closures `heat_transfer_study.py:L2534-L3292`; `heat_transfer_study.py:L4326-L4438`.
3. A Python annual layer reuses the selected heating and cooling reference points against a representative-year ambient driver `heat_transfer_study.py:L4547-L5012`; `heat_transfer_study.py:L5013-L5496`.

The physically important paths are electrical Joule heating of the wire, wire-to-air convection, heater-shell loss to ambient, sensible air-to-bed transfer through the aeration tube wall, direct discharge enthalpy through the perforations, latent heat consumption by evaporation, and bed-to-ambient loss through the vessel wall and top boundary condition.

This document is intentionally more explicit than `HeatTransferStudy/VermiHeatLoss.tex`. It restates the implemented equation families, identifies where they appear in the repository narrative, and records the exact code locations where each block is implemented or approximated.

2 Modeling boundaries and scope

- MATLAB is the authoritative source of the winter/heating operating-point grid and geometry-driven heater calculations `vermicomposter_air_heater_design_model.m:L1172-L1677`.
- Python is authoritative for the final reported heating recommendation, the full summer cooling workflow, the annual layer, and the current text summaries and plots `heat_transfer_study.py:L2220-L2499`; `heat_transfer_study.py:L2534-L3292`; `heat_transfer_study.py:L5013-L7020`.
- The model is reduced-order. Tubes are one-dimensional marching domains; the bed is a two-node lumped thermal network; plume and habitat calculations are post-processing estimates rather than CFD `vermicomposter_air_heater_design_model.m:L1803-L2248`; `heat_transfer_study.py:L1062-L1595`.
- The summer branch reuses the same bin wall, aeration-tube, evaporation, and two-node closures,

but replaces electrical heating with spot-cooled inlet air and an assist-blower static constraint `heat_transfer_study.py:L2605-L3292`.

- This reference documents only equations actually found in the repository. Where `HeatTransferStudy/VermiHeatLoss.tex` contains broader narrative or optional transient discussion not used by the rooted execution graph, that is called out explicitly.

3 Modeling assumptions

- Air is treated as an ideal gas with Sutherland-law viscosity and fixed c_p and Pr in the property package `heat_transfer_study.py:L3607-L3675`; `vermicomposter_air_heater_design_model.m:L2333-L2359`.
- Heater and aeration tubes are discretized axially and solved by explicit marching rather than simultaneous PDE solution `vermicomposter_air_heater_design_model.m:L1803-L1999`; `heat_transfer_study.py:L2707-L2959`.
- Parallel aeration branches are identical at a given operating point. Total heat, water loss, and power are assembled from per-tube or per-string contributions `vermicomposter_air_heater_design_model.m:L1497-L1677`.
- The bed is represented by bottom and top thermal nodes with a single internal conductance between them `vermicomposter_air_heater_design_model.m:L971-L1075`; `heat_transfer_study.py:L4326-L4438`.
- Evaporation is represented as a local latent sink limited by diffusion, carrying capacity, and available sensible heat; there is no resolved liquid transport field in the compost `vermicomposter_air_heater_design_model.m:L2000-L2122`; `heat_transfer_study.py:L2567-L2603`.
- The vented-cover top is a repository-specific mixed boundary condition: insulated cover over most of the top plus free convection over a small vent area `vermicomposter_air_heater_design_model.m:L971-L1075`; `HeatTransferStudy/VermiHeatLoss.tex:L785-L847`.
- Summer spot-cooler behavior is a surrogate model based on configured rated quantities and simple psychrometric limiting, not a first-principles vapor-compression cycle model `heat_transfer_study.py:L2605-L2705`.

4 Energy balances

4.1 Air and psychrometric property package

Dry-air density is evaluated as

$$\rho = \frac{p}{RT}. \tag{1}$$

Dynamic viscosity uses the Sutherland form

$$\mu = \mu_0 \left(\frac{T}{T_0}\right)^{3/2} \frac{T_0 + S}{T + S}. \tag{2}$$

Thermal conductivity is then reconstructed from

$$k = \frac{\mu c_p}{Pr}. \tag{3}$$

For the summer psychrometric branch, humidity ratio is computed from vapor partial pressure p_v as

$$w = 0.62198 \frac{p_v}{p - p_v}, \tag{4}$$

and, for a given relative humidity ϕ,

$$p_v = \phi\, p_{\mathrm{sat}}(T). \tag{5}$$

Type. Equation (**??**) is exact for the ideal-gas approximation adopted here; (**??**) is a fitted transport-property relation; (**??**) is a derived closure using fixed c_p and Pr; (**??**)–(**??**) are exact algebraic psychrometric relations for the implemented moisture basis.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L11-L37`; `HeatTransferStudy/VermiHeatLoss.tex:L1089-L1235`; `HeatTransferStudy/VermiHeatLoss.tex:L2084-L2245`.
Implementation. `vermicomposter_air_heater_design_model.m:L2333-L2359`; `heat_transfer_study.py:L3607-L3708`.
Deviation note. The property package uses constant c_p and Pr rather than temperature-dependent tabulated humid-air properties.

4.2 Tube-flow partitions, perforations, and electrical grouping

Let the total imposed inlet volumetric flow be $\dot{V}_{\mathrm{tot}}$ and the number of parallel aeration branches be N_{tubes}. The per-tube inlet volumetric flow is

$$\dot{V}_{\mathrm{tube,in}} = \frac{\dot{V}_{\mathrm{tot}}}{N_{\mathrm{tubes}}}. \tag{6}$$

Using the inlet density ρ_{in}, the corresponding per-tube inlet mass flow is

$$\dot{m}_{\mathrm{tube,in}} = \rho_{\mathrm{in}} \dot{V}_{\mathrm{tube,in}}. \tag{7}$$

If a fraction ϕ_{rel} is released through perforations,

$$\dot{m}_{\mathrm{rel}} = \phi_{\mathrm{rel}} \dot{m}_{\mathrm{tube,in}}. \tag{8}$$

For N_h holes of diameter d_h,

$$A_h = \frac{\pi d_h^2}{4}, \qquad A_{h,\mathrm{tot}} = N_h A_h, \tag{9}$$

and the mean release velocity is

$$u_h = \frac{\dot{V}_{\mathrm{rel}}}{N_h A_h}. \tag{10}$$

The coil helix geometry is built from the mean coil diameter D_{mean}, pitch p, and axial coil span L_{coil}. The per-turn length is

$$L_{\mathrm{turn}} = \sqrt{(\pi D_{\mathrm{mean}})^2 + p^2}, \tag{11}$$

and the wire length is

$$L_{\mathrm{wire}} = N_{\mathrm{turns}} L_{\mathrm{turn}}, \qquad N_{\mathrm{turns}} \approx \frac{L_{\mathrm{coil}}}{p}. \tag{12}$$

With wire resistivity $\rho_{\mathrm{wire}}(T)$ and cross-sectional area A_{wire},

$$R_{\mathrm{tube}} = \rho_{\mathrm{wire}}(T_{\mathrm{avg}}) \frac{L_{\mathrm{wire}}}{A_{\mathrm{wire}}}. \tag{13}$$

The current heating implementation supports groups of series-connected tubes in parallel strings. For group g, let $n_{s,g}$ be the number of tubes in series per string and $n_{\mathrm{str},g}$ the number of such strings. Then

$$I_{\mathrm{tube},g} = \frac{V}{n_{s,g} R_{\mathrm{tube}}}, \tag{14}$$

$$P_{\mathrm{tube},g} = I_{\mathrm{tube},g}^2 R_{\mathrm{tube}}, \tag{15}$$

and with $n_{\mathrm{tubes},g} = n_{s,g} n_{\mathrm{str},g}$,

$$I_{\mathrm{tot}} = \sum_g n_{\mathrm{str},g} I_{\mathrm{tube},g}, \qquad P_{\mathrm{tot}} = \sum_g n_{\mathrm{tubes},g} P_{\mathrm{tube},g}. \tag{16}$$

The heater-tube air inlet is the greenhouse ambient state,

$$T_{\mathrm{air,in}} = T_\infty, \tag{17}$$

and the aeration-tube inlet is the heater-tube outlet,

$$T_{\mathrm{air,in,aer}} = T_{\mathrm{air,out,heater}}. \tag{18}$$

For any tube of outer diameter D_o, the outside perimeter used to convert an axial length to an external heat-transfer area is

$$P_{\mathrm{out}} = \pi D_o, \qquad A_{\mathrm{seg,out}} = P_{\mathrm{out}} \Delta x. \tag{19}$$

Type. Equations (**??**)–(**??**) are exact algebraic bookkeeping relations on the reduced-order model state.
Repository-native narrative. The same physical quantities appear in `HeatTransferStudy/VermiHeatLoss.tex:L961-L1034`, `HeatTransferStudy/VermiHeatLoss.tex:L1560-L1738`, and `HeatTransferStudy/VermiHeatLoss.tex:L2302-L2444`.
Implementation. `vermicomposter_air_heater_design_model.m:L763-L875`; `vermicomposter_air_heater_design_model.m:L1497-L1677`; `vermicomposter_air_heater_design_model.m:L1684-L1727`.
Deviation note. The MATLAB implementation permits mixed string sizes for odd tube counts; the equations above are therefore written in grouped summation form rather than assuming a single equivalent series-parallel pattern.

4.3 Heater-tube energy balance

With heater length L_{heater} discretized into N_{heater} segments,

$$\Delta x_{\mathrm{heater}} = \frac{L_{\mathrm{heater}}}{N_{\mathrm{heater}}}, \tag{20}$$

and the local generated electrical heat assigned to segment i is

$$q_{\mathrm{gen},i} = P_{\mathrm{branch}} \frac{\Delta x_{\mathrm{heater}}}{L_{\mathrm{heater}}}. \tag{21}$$

For heater segment i, the air-energy update is

$$T_{\mathrm{air},i+1} = T_{\mathrm{air},i} + \frac{q_{\mathrm{gen},i} - q_{\mathrm{loss},i}}{\dot{m} c_p}. \tag{22}$$

Wire temperature is then recovered from the local wire-to-air temperature rise

$$\Delta T_{\mathrm{wire},i} = \frac{q_{\mathrm{gen},i}}{h_{\mathrm{wire},i} A'_{\mathrm{wire}} \Delta x}, \qquad T_{\mathrm{wire},i} = T_{\mathrm{air},i} + \Delta T_{\mathrm{wire},i}. \tag{23}$$

The segmentwise heater-shell loss is

$$q_{\mathrm{loss},i} = U_{\mathrm{heater}} P_{\mathrm{out}} \Delta x_{\mathrm{heater}} \left(T_{\mathrm{air},i} - T_{\infty} \right). \tag{24}$$

Type. Equation (**??**) is exact segment bookkeeping on the reduced model. Equation (**??**) is a discretized one-dimensional steady energy balance. Equations (**??**) and (**??**) are reduced-order algebraic temperature-drop and shell-loss closures.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L38-L121`.
Implementation. `vermicomposter_air_heater_design_model.m:L1803-L1894`; `heat_transfer_study.py:L3775-L3895`.
Deviation note. The wire-to-air temperature difference is represented by a direct convective drop rather than a resolved wire conduction model.

4.4 Aeration-tube segment balance

With aeration length L_{aer} discretized into N_{aer} segments,

$$\Delta x_{\mathrm{aer}} = \frac{L_{\mathrm{aer}}}{N_{\mathrm{aer}}}, \tag{25}$$

and with a uniform segmentwise released mass,

$$\dot{m}_{\mathrm{rel},i} = \frac{\phi_{\mathrm{rel}} \dot{m}_{\mathrm{tube,in}}}{N_{\mathrm{aer}}}. \tag{26}$$

For aeration segment i, the net bed-side energy contribution is

$$q_{\mathrm{bed},i} = q_{\mathrm{wall},i} + q_{\mathrm{jet},i} - q_{\mathrm{evap},i}. \tag{27}$$

Summing over the segment set gives

$$Q_{\mathrm{bed,branch}} = \sum_i q_{\mathrm{bed},i}, \tag{28}$$

and the total bed heat rate across N_{tubes} identical branches is

$$Q_{\mathrm{bed,total}} = N_{\mathrm{tubes}} Q_{\mathrm{bed,branch}}. \tag{29}$$

The winter MATLAB march reduces the in-tube mass flow after each segment by

$$\dot{m}_{\mathrm{rem},i+1} = \dot{m}_{\mathrm{rem},i} - \dot{m}_{\mathrm{rel},i}, \tag{30}$$

and advances the bulk in-tube air temperature using only the wall-transfer term,

$$T_{\mathrm{air},i+1} = T_{\mathrm{air},i} - \frac{q_{\mathrm{wall},i}}{\dot{m}_{\mathrm{rem},i+1} c_p}. \tag{31}$$

The summer Python branch instead applies the same wall exchange through an exponential segment factor,

$$\Theta_i = \exp\left[-\frac{U_{\text{aer}} P_{\text{out}} \Delta x_{\text{aer}}}{\dot{m}_{\text{mean},i} c_p}\right], \tag{32}$$

$$T_{\text{air,after wall},i} = T_{\text{bed,ref}} + \left(T_{\text{air},i} - T_{\text{bed,ref}}\right)\Theta_i, \tag{33}$$

with

$$q_{\text{wall},i} = \dot{m}_{\text{mean},i} c_p \left(T_{\text{air},i} - T_{\text{air,after wall},i}\right). \tag{34}$$

Type. Derived reduced-order balance.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L151-L205`; `HeatTransferStudy/VermiHeatLoss.tex:L1586-L1609`; `HeatTransferStudy/VermiHeatLoss.tex:L2168-L2238`.
Implementation. `vermicomposter_air_heater_design_model.m:L1895-L1999`; `heat_transfer_study.py:L2821-L2899`.
Deviation note. The model treats released-air sensible exchange as a lumped enthalpy source rather than resolving jets in the pore space. The rooted execution graph therefore uses two closely related but not identical march forms: a winter MATLAB explicit wall-only air update and a summer Python exponential wall-factor update.

4.5 Two-node bed balance

The bottom and top bed nodes are solved from the steady 2×2 linear system

$$\begin{bmatrix} UA_{b,\infty} + UA_{12} & -UA_{12} \\ -UA_{12} & UA_{t,\infty} + UA_{12} \end{bmatrix} \begin{bmatrix} T_b \\ T_t \end{bmatrix} = \begin{bmatrix} Q_b + UA_{b,\infty} T_\infty \\ Q_t + UA_{t,\infty} T_\infty \end{bmatrix}, \tag{35}$$

with

$$Q_b = f_b Q_{\text{bed,total}}, \qquad Q_t = (1 - f_b) Q_{\text{bed,total}}. \tag{36}$$

Type. Derived algebraic reduction of a two-node thermal network.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L365-L424`; `HeatTransferStudy/VermiHeatLoss.tex:L2010-L2079`.
Implementation. `vermicomposter_air_heater_design_model.m:L2233-L2248`; `heat_transfer_study.py:L4432-L4438`.
Deviation note. The current rooted execution path does not use the optional transient MATLAB PID simulation; the authoritative audit path is the steady algebraic solve above.

5 Fluid mechanics and pressure-drop model

The code constructs Reynolds number in the standard form

$$Re = \frac{\rho u D}{\mu}. \tag{37}$$

For laminar internal flow, the Darcy friction factor is

$$f = \frac{64}{Re}. \tag{38}$$

For turbulent internal flow, the implemented smooth-tube Churchill expression is

$$A = \left[2.457 \ln\left(\frac{1}{(7/Re)^{0.9}}\right)\right]^{16}, \qquad B = \left(\frac{37530}{Re}\right)^{16}, \tag{39}$$

$$f = 8\left[\left(\frac{8}{Re}\right)^{12} + \frac{1}{(A+B)^{3/2}}\right]^{1/12}. \tag{40}$$

Tube friction losses are accumulated with Darcy–Weisbach:

$$\Delta p_f = f\frac{L}{D}\frac{\rho u^2}{2}. \tag{41}$$

Abrupt expansion is modeled as

$$\zeta_{\text{exp}} = \left(1 - \frac{A_1}{A_2}\right)^2, \qquad \Delta p_{\text{exp}} = \zeta_{\text{exp}}\frac{\rho u_1^2}{2}. \tag{42}$$

The conical contraction surrogate uses an Idel'chik bellmouth coefficient $C(\epsilon, \theta)$ and applies

$$\zeta_{\text{contr}} = C(\epsilon, \theta)\left(\frac{1}{\epsilon} - 1\right), \qquad \Delta p_{\text{contr}} = \zeta_{\text{contr}}\frac{\rho u_2^2}{2}. \tag{43}$$

The distribution-network losses are then assembled as

$$\Delta p_{\text{dist}} = \Delta p_{\text{header}} + \Delta p_{\text{header}\to\text{splitter}} + \Delta p_{\text{splitter}} + \Delta p_{\text{connector,fric}} + \Delta p_{\text{connector}\to\text{branch}}. \tag{44}$$

The total branch-plus-distribution drop is

$$\Delta p_{\text{tot}} = \Delta p_{\text{branch}} + \Delta p_{\text{dist}}. \tag{45}$$

Type. Equations (**??**)–(**??**) are standard fluid-mechanics relations plus a repository-configured minor-loss assembly.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L862-L961`; `HeatTransferStudy/VermiHeatLoss.tex:L1899-L2009`.
Implementation. `vermicomposter_air_heater_design_model.m:L2123-L2232`; `vermicomposter_air_heater_design_model.m:L2399-L2407`; `heat_transfer_study.py:L3755-L3763`; `heat_transfer_study.py:L3961-L4319`.
Deviation note. The distribution hardware beyond the measured geometry is partly represented by editable lumped K-values, especially the splitter body loss.

6 Internal convection correlations

The internal convection coefficient is always formed from

$$h = \frac{Nu\, k}{D}. \tag{46}$$

For laminar developing flow, the implemented Hausen-type relation is

$$Gz = Re\, Pr\, \frac{D}{L}, \tag{47}$$

$$Nu = 3.66 + \frac{0.0668\,Gz}{1 + 0.04\,Gz^{2/3}}, \qquad Re < 2300. \tag{48}$$

For turbulent flow, the implemented Gnielinski relation is

$$Nu = \max\left[3.66,\ \frac{(f/8)(Re - 1000)Pr}{1 + 12.7(f/8)^{1/2}(Pr^{2/3} - 1)}\right], \qquad Re \geq 2300. \tag{49}$$

Type. Empirical correlations.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L80-L103`; `HeatTransferStudy/VermiHeatLoss.tex:L862-L961`.
Implementation. `vermicomposter_air_heater_design_model.m:L2382-L2398`; `heat_transfer_study.py:L3764-L3774`.
Deviation note. The same branch logic is used for both heater and aeration tubes. No separate roughness correction or transitional blending beyond the branch cutoff is implemented.

7 External convection correlations

For crossflow around the heater wire and perforation-scale characteristic diameter, the code uses the Churchill–Bernstein correlation

$$Nu = 0.3 + \frac{0.62Re^{1/2}Pr^{1/3}}{\left[1 + (0.4/Pr)^{2/3}\right]^{1/4}}\left[1 + \left(\frac{Re}{282000}\right)^{5/8}\right]^{4/5}. \tag{50}$$

For external vertical walls, the Rayleigh number is

$$Ra = \frac{g\beta\Delta TL^3}{\nu\alpha}, \tag{51}$$

and the Churchill–Chu plate relation is

$$Nu = \left[0.825 + \frac{0.387Ra^{1/6}}{\left(1 + (0.492/Pr)^{9/16}\right)^{8/27}}\right]^2, \qquad h = \frac{Nuk}{L}. \tag{52}$$

For horizontal top or vent surfaces, the implemented piecewise free-convection relation is

$$Nu = \begin{cases} 0.54Ra^{1/4}, & T_s \geq T_\infty,\ Ra \leq 10^7, \\ 0.15Ra^{1/3}, & T_s \geq T_\infty,\ Ra > 10^7, \\ 0.27Ra^{1/4}, & T_s < T_\infty. \end{cases} \tag{53}$$

Type. Empirical correlations.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L30-L57`; `HeatTransferStudy/VermiHeatLoss.tex:L203-L244`; `HeatTransferStudy/VermiHeatLoss.tex:L862-L961`.
Implementation. `vermicomposter_air_heater_design_model.m:L1111-L1155`; `vermicomposter_air_heater_design_model.m:L2372-L2381`; `heat_transfer_study.py:L3710-L3754`.
Deviation note. The same Churchill–Bernstein form is reused both for wire cooling and for hole-scale evaporation coupling.

8 Conduction and thermal resistance networks

The implemented wall-stack coefficient is

$$U_{\text{wall}} = \left(\frac{1}{h_{\text{inside,bin}}} + \frac{t_{\text{sheet}}}{k_{\text{sheet}}} + \frac{t_{\text{ins}}}{k_{\text{ins}}} + \frac{1}{h_{\text{ext}}} \right)^{-1}. \tag{54}$$

The same algebraic structure is used for the heater tube and aeration tube:

$$U_{\text{heater}} = \left(\frac{1}{h_{\text{inside,heater}}} + \frac{D_o - D_i}{2k_{\text{wall}}} + \frac{t_{\text{ins}}}{k_{\text{ins}}} + \frac{1}{h_{\text{outside,heater}}} \right)^{-1}, \tag{55}$$

$$U_{\text{aer}} = \left(\frac{1}{h_{\text{inside,aer}}} + \frac{D_o - D_i}{2k_{\text{wall}}} + \frac{1}{h_{\text{outside,aer}}} \right)^{-1}. \tag{56}$$

Type. Simplified thermal resistance networks.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L103-L200`; `HeatTransferStudy/VermiHeatLoss.tex:L203-L244`.
Implementation. `vermicomposter_air_heater_design_model.m:L1102-L1110`; `vermicomposter_air_heater_design_model.m:L1884-L1894`; `heat_transfer_study.py:L3737-L3778`.
Deviation note. The radial wall terms use thickness-over-conductivity approximations rather than exact cylindrical $\ln(r_o/r_i)$ resistances. That simplification is in the implementation and is therefore documented here as implemented.

9 Heater-side overall heat transfer treatment

The wire-side convective coefficient is

$$h_{\text{wire}} = \frac{Nu_{\text{wire}} k}{D_{\text{wire}}}, \tag{57}$$

with Nu_{wire} given by (**??**). The heater-shell loss per segment is

$$q_{\text{loss},i} = U_{\text{heater}} P_o \Delta x \left(T_{\text{air},i} - T_\infty \right), \tag{58}$$

and the local net air heating is

$$q_{\text{to air},i} = q_{\text{gen},i} - q_{\text{loss},i}. \tag{59}$$

Type. Derived reduced-order heater balance.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L38-L121`; `HeatTransferStudy/VermiHeatLoss.tex:L1738-L1898`.
Implementation. `vermicomposter_air_heater_design_model.m:L1803-L1894`; `heat_transfer_study.py:L3775-L3895`.
Deviation note. The Python physics re-solver mirrors the MATLAB heater equations for annual and diagnostic use, but the winter operating grid itself still originates from MATLAB.

10 Aeration-tube heat transfer model

The wall-mediated sensible contribution is

$$q_{\mathrm{wall},i} = U_{\mathrm{aer}} P_o \Delta x \left(T_{\mathrm{air},i} - T_{\mathrm{bed,ref}}\right). \tag{60}$$

The direct released-air enthalpy term is

$$q_{\mathrm{jet},i} = \dot{m}_{\mathrm{rel},i} c_p \left(T_{\mathrm{air},i} - T_{\mathrm{bed,ref}}\right). \tag{61}$$

The in-tube air-temperature march is not a single universal formula across the rooted execution graph. The winter MATLAB branch uses the explicit wall-only update in (**??**), whereas the summer Python branch uses the exponential wall-factor form (**??**) with the exact wall exchange (**??**). In both cases, the net bed contribution still follows (**??**), so the sign of $q_{\mathrm{jet},i}$ and of the total segment contribution naturally reverses between heating and cooling.

Type. Derived reduced-order segment model.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L151-L205`; `HeatTransferStudy/VermiHeatLoss.tex:L2168-L2238`.
Implementation. `vermicomposter_air_heater_design_model.m:L1895-L1999`; `heat_transfer_study.py:L2815-L2959`.
Deviation note. The air-release fraction is prescribed, not dynamically solved from local perforation pressure and hole discharge coefficients.

11 Warm-air release / jet-heating treatment

The released-air term (**??**) is the model's surrogate for direct pore-space heating or cooling by discharged air. It should be interpreted as an enthalpy source or sink attached to the released fraction $\dot{m}_{\mathrm{rel},i}$, not as a resolved jet correlation. In the summer branch, the same equation is reused with $T_{\mathrm{air}} < T_{\mathrm{bed,ref}}$, so $q_{\mathrm{jet},i}$ becomes negative and contributes to net cooling. The associated radial plume cooling, worm-safe distance, and habitat-exclusion mathematics are documented in full in the later plume/habitat section beginning at (**??**).

Type. Reduced-order enthalpy model plus repository-specific plume post-processing.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L1342-L1585`.
Implementation. `heat_transfer_study.py:L1062-L1306`; `heat_transfer_study.py:L1686-L1736`.
Deviation note. The plume model is not used to modify the main tube-marching solution; it is a separate post-processing estimate.

12 Moisture, evaporation, and latent-heat treatment

The current wetted-area basis can be the top plan area, a manual area, or a particle-surface estimate derived from the Table 3 size fractions. For the particle-surface path, the Sauter mean diameter is

$$\frac{1}{d_{32}} = \sum_i \frac{w_i}{d_i}, \tag{62}$$

so the external surface area per occupied bed volume is

$$a_{\text{ext}} = \frac{6}{d_{32}}. \tag{63}$$

If V_{fill} is the filled bed volume and f_{access} the accessible wet-surface fraction,

$$A_{\text{wet}} = f_{\text{access}} a_{\text{ext}} V_{\text{fill}}. \tag{64}$$

The saturation property package uses IAPWS ancillary relations. The saturation pressure is represented as

$$\ln\left(\frac{p_{\text{sat}}}{p_c}\right) = \frac{T_c}{T}\sum_{i=1}^{6} a_i \tau^{b_i}, \qquad \tau = 1 - \frac{T}{T_c}, \tag{65}$$

with analogous ancillary forms for $\rho_{\ell,\text{sat}}$ and $\rho_{v,\text{sat}}$. The latent heat is then computed by Clausius–Clapeyron along saturation:

$$h_{fg}(T) = T\frac{dp_{\text{sat}}}{dT}\left(\frac{1}{\rho_{v,\text{sat}}(T)} - \frac{1}{\rho_{\ell,\text{sat}}(T)}\right). \tag{66}$$

The hole-scale convective coefficient is formed from (**??**) and converted to a mass-transfer coefficient by the Lewis relation:

$$h_m = \frac{h_{\text{evap}}}{\rho c_p Le^{2/3}}. \tag{67}$$

The vapor-density driving force is

$$\Delta\rho_v = \max[\rho_{v,\text{sat}}(T_s) - \rho_{v,\infty},\, 0]\,. \tag{68}$$

The diffusion-limited, carrying-capacity-limited, and heat-limited evaporation rates are

$$\dot{m}_{\text{diff}} = h_m A_{\text{seg}} \Delta\rho_v, \tag{69}$$

$$\dot{m}_{\text{cap}} = \dot{V}_{\text{rel,seg}} \Delta\rho_v, \tag{70}$$

$$\dot{m}_{\text{heat}} = \frac{q_{\text{avail}}}{h_{fg}}, \qquad q_{\text{avail}} = \max[\dot{m}_{\text{rel,seg}} c_p (T_{\text{air}} - T_{\text{bed}}),\, 0]\,. \tag{71}$$

The implemented segment evaporation rate and latent sink are

$$\dot{m}_{\text{evap,seg}} = \max[0,\, \min(\dot{m}_{\text{diff}}, \dot{m}_{\text{cap}}, \dot{m}_{\text{heat}})]\,, \qquad q_{\text{evap,seg}} = \dot{m}_{\text{evap,seg}} h_{fg}. \tag{72}$$

Type. Combination of fitted water-property relations, empirical transport relations, and repository-specific area bookkeeping.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L1015-L1369`; `HeatTransferStudy/VermiHeatLoss.tex:L1586-L1609`; `HeatTransferStudy/VermiHeatLoss.tex:L2238-L2245`.
Implementation. `vermicomposter_air_heater_design_model.m:L891-L937`; `vermicomposter_air_heater_design_model.m:L2000-L2122`; `heat_transfer_study.py:L2567-L2603`; `heat_transfer_study.py:L3607-L3708`.
Deviation note. The summer branch computes inlet humidity after spot-cooler condensate removal, so the effective $\rho_{v,\infty}$ is not always the raw ambient-RH state.

13 Radial plume, worm-safe stand-off, and habitat-exclusion post-processing

The released-air post-processing model starts from the released mass flow rate per unit aeration-tube length,

$$\dot{m}'_{\mathrm{rel}} = \frac{\phi_{\mathrm{rel}} \dot{m}_{\mathrm{tube,in}}}{L_{\mathrm{aer}}}, \tag{73}$$

and the wetted-area density

$$a_{\mathrm{wet}} = \frac{A_{\mathrm{wet}}}{V_{\mathrm{fill}}}. \tag{74}$$

The local wet-surface sink temperature is clamped between the configured surface temperature, the bed-reference temperature, and the released-air temperature:

$$T_{\mathrm{sink}} = \min(\max(T_{s,\mathrm{set}}, T_{\mathrm{bed,ref}}), T_{\mathrm{air,out}}) \,. \tag{75}$$

The plume radius is grown as

$$b(x) = \min(b_{\max},\, b_0 + \alpha x) \,, \tag{76}$$

so the contacted wetted area per radial length is

$$A'_{\mathrm{wet}}(x) = \pi a_{\mathrm{wet}} b(x)^2. \tag{77}$$

The sensible and latent radial sink terms are

$$q'_{\mathrm{sens}}(x) = h_{\mathrm{sens}} A'_{\mathrm{wet}}(x) \left(T - T_{\mathrm{sink}}\right), \tag{78}$$

$$q'_{\mathrm{lat}}(x) = \dot{m}'_{\mathrm{evap}}(x) h_{fg}(T_s), \tag{79}$$

and the plume temperature satisfies

$$\dot{m}'_{\mathrm{rel}} c_p \frac{dT}{dx} = -\left[q'_{\mathrm{sens}}(x) + q'_{\mathrm{lat}}(x)\right]. \tag{80}$$

The local mass-transfer coefficient is formed by the Lewis relation using local air properties,

$$h_m(x) = \frac{h_{\mathrm{evap}}}{\rho_{\mathrm{air}}(T) c_{p,\mathrm{air}} \mathrm{Le}^{2/3}}, \tag{81}$$

which gives the potential diffusion-driven evaporation rate per radial length,

$$\dot{m}'_{\mathrm{evap,pot}} = h_m(x) A'_{\mathrm{wet}}(x) \left[\rho_{v,\mathrm{sat}}(T_s) - \rho_{v,\mathrm{bulk}}(x)\right]. \tag{82}$$

The code advances the plume humidity explicitly and caps the bulk vapor density at saturation at the updated plume temperature `heat_transfer_study.py:L1241-L1292`. If the worm-safety threshold is T_{safe}, the safe radial offset is the first x such that

$$T(x) = T_{\mathrm{safe}}, \tag{83}$$

located in the code by interpolation on the numerically marched profile `heat_transfer_study.py:L1307-L1332`.

For habitat accounting, the filled-bed height and volume are

$$H_{\mathrm{fill}} = f_{\mathrm{fill}} H, \qquad V_{\mathrm{fill}} = L W H_{\mathrm{fill}}, \tag{84}$$

with filled-bed cross section

$$\Omega_{\text{fill}} = \{(x, y) : 0 \le x \le W,\ 0 \le y \le H_{\text{fill}}\}\,. \tag{85}$$

If the safe offset is x_{safe}, the exclusion radius around each tube is

$$R_{\text{exc}} = r_0 + x_{\text{safe}}. \tag{86}$$

The pipe and exclusion indicator functions are

$$I_{\text{pipe}}(x, y) = \begin{cases} 1, & \exists i:\ (x - x_i)^2 + (y - y_i)^2 \le r_0^2, \\ 0, & \text{otherwise}, \end{cases} \tag{87}$$

$$I_{\text{exc}}(x, y) = \begin{cases} 1, & \exists i:\ (x - x_i)^2 + (y - y_i)^2 \le R_{\text{exc}}^2, \\ 0, & \text{otherwise}. \end{cases} \tag{88}$$

The overlap-aware union areas are

$$A_{\text{pipe,union}} = \iint_{\Omega_{\text{fill}}} I_{\text{pipe}}(x, y)\, dA, \tag{89}$$

$$A_{\text{exc,union}} = \iint_{\Omega_{\text{fill}}} I_{\text{exc}}(x, y)\, dA, \tag{90}$$

with volumes obtained by extrusion over aeration-tube length,

$$V_{\text{pipe}} = A_{\text{pipe,union}} L_{\text{aer}}, \qquad V_{\text{exc}} = A_{\text{exc,union}} L_{\text{aer}}. \tag{91}$$

The shell-only unsafe volume and remaining habitable volume are then

$$V_{\text{unsafe,shell}} = V_{\text{exc}} - V_{\text{pipe}}, \tag{92}$$

$$V_{\text{hab}} = V_{\text{fill}} - V_{\text{unsafe,total}}. \tag{93}$$

When the radial plume reaches the left, right, or bottom wall, the code computes an inside-wall temperature from the same wall stack used in the main bin-loss model,

$$q''_{\text{wall}} = \frac{T_{\text{plume,wall}} - T_\infty}{\frac{1}{h_{\text{in}}} + \frac{t_{\text{sheet}}}{k_{\text{sheet}}} + \frac{t_{\text{ins}}}{k_{\text{ins}}} + \frac{1}{h_{\text{ext}}}}, \tag{94}$$

$$T_{\text{wall,in}} = T_{\text{plume,wall}} - \frac{q''_{\text{wall}}}{h_{\text{in}}}, \tag{95}$$

then converts that to an added unsafe band depth with a linear wall-normal interpolation between the bed temperature and the wall-contact temperature `heat_transfer_study.py:L1531-L1555`. The final reported unsafe volume is the union of plume exclusion cylinders and wall-return bands:

$$V_{\text{unsafe,total}} = V_{\text{union}}. \tag{96}$$

For overlap reporting, the naive no-overlap sums are

$$A_{\text{pipe,sum}} = N_{\text{tubes}} \pi r_0^2, \qquad A_{\text{exc,sum}} = N_{\text{tubes}} \pi R_{\text{exc}}^2, \tag{97}$$

with overlap corrections

$$A_{\text{pipe,ov}} = A_{\text{pipe,sum}} - A_{\text{pipe,union}}, \qquad A_{\text{exc,ov}} = A_{\text{exc,sum}} - A_{\text{exc,union}}. \tag{98}$$

Numerically, the union areas are evaluated on a midpoint grid:

$$A \approx \left(\frac{WH_{\text{fill}}}{N_g^2}\right) \sum_{j=1}^{N_g} \sum_{k=1}^{N_g} I(x_j^*, y_k^*). \tag{99}$$

Type. Repository-specific radial thermal/moisture post-processing and overlap-aware geometry bookkeeping.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L1277-L1571`.
Implementation. `heat_transfer_study.py:L1119-L1367`; `heat_transfer_study.py:L1440-L1664`.
Deviation note. The plume model is not part of the core heater or cooling-march solve. It is a post-processing estimator used for the radial cooling plot and the habitat-exclusion metrics.

14 Bin-wall and ambient heat-loss model

The vessel geometry is

$$A_{\text{side}} = 2HL + 2HW, \qquad A_{\text{bottom}} = LW, \qquad A_{\text{top}} = LW. \tag{100}$$

With N_{vent} top vent holes of diameter d_{vent},

$$A_{\text{vent}} = N_{\text{vent}} \frac{\pi d_{\text{vent}}^2}{4}, \qquad A_{\text{top,cov}} = A_{\text{top}} - A_{\text{vent}}. \tag{101}$$

For open-top convection, the characteristic length is taken as

$$L_{c,\text{top}} = \frac{A_{\text{top}}}{P_{\text{top}}}, \qquad P_{\text{top}} = 2(L + W), \tag{102}$$

so the top natural-convection coefficient is

$$h_{\text{top}} = \frac{Nu_{\text{top}} k}{L_{c,\text{top}}}, \tag{103}$$

with Nu_{top} given by (**??**). The total vessel heat-loss forms can then be written explicitly as

$$Q_{\text{loss,closed}} = U_{\text{wall}}(A_{\text{side}} + A_{\text{bottom}} + A_{\text{top}})(T_{\text{bin}} - T_\infty), \tag{104}$$

$$Q_{\text{loss,open}} = U_{\text{wall}}(A_{\text{side}} + A_{\text{bottom}})(T_{\text{bin}} - T_\infty) + h_{\text{top}} A_{\text{top}}(T_{\text{bin}} - T_\infty). \tag{105}$$

For an open top, the top-node and total ambient conductances are

$$UA_{\text{top,open}} = U_{\text{wall}} A_{\text{top,wallshare}} + h_{\text{top}} A_{\text{top}}, \tag{106}$$

$$UA_{\text{tot,open}} = U_{\text{wall}}(A_{\text{side}} + A_{\text{bottom}}) + h_{\text{top}} A_{\text{top}}. \tag{107}$$

For a covered top with a small vent,

$$UA_{\text{top,vented}} = U_{\text{wall}}(A_{\text{top,cov}} + A_{\text{top,wallshare}}) + h_{\text{top,vent}} A_{\text{vent}}, \tag{108}$$

$$UA_{\text{tot,vented}} = U_{\text{wall}}(A_{\text{side}} + A_{\text{bottom}} + A_{\text{top,cov}}) + h_{\text{top,vent}} A_{\text{vent}}. \tag{109}$$

For a fully covered top,

$$UA_{\text{tot,closed}} = U_{\text{wall}}(A_{\text{side}} + A_{\text{bottom}} + A_{\text{top}}). \tag{110}$$

The lumped required heating or cooling load is

$$Q_{\text{req}} = UA_{\text{tot}} \left(T_{\text{set}} - T_{\infty}\right). \tag{111}$$

The characteristic length and Biot estimate are

$$L_c = \frac{V_{\text{fill}}}{A_{\text{side}} + A_{\text{bottom}} + A_{\text{top}}}, \qquad Bi = \frac{h_{\text{ext}} L_c}{k_{\text{bed}}}. \tag{112}$$

The lumped time constant is

$$\tau = \frac{C}{UA_{\text{tot}}}, \tag{113}$$

and the associated cooldown curve is

$$T(t) = T_{\infty} + (T_0 - T_{\infty})\, e^{-t/\tau}. \tag{114}$$

Type. Derived thermal-network relations plus a repository-specific vented-cover extension.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L603-L847`.
Implementation. `vermicomposter_air_heater_design_model.m:L971-L1171`; `heat_transfer_study.py:L4326-L4431`.
Deviation note. The rooted execution path only plots the lumped cooldown curve when $Bi < 0.1$; the equation exists regardless, but the code treats the plot as conditionally valid rather than unconditional.

14.1 Summer spot-cooler and assist-blower surrogate

In the summer branch, the requested sensible cooling duty for target supply temperature $T_{\text{supply,target}}$ is

$$\dot{Q}_{\text{req,cool}} = \max[\dot{m} c_p \left(T_{\text{amb}} - T_{\text{supply,target}}\right), 0]. \tag{115}$$

The actual sensible cooling applied by the spot cooler is capped at the configured or derived sensible capacity:

$$\dot{Q}_{\text{actual,cool}} = \min\Big(\dot{Q}_{\text{req,cool}}, \dot{Q}_{\text{max,cool}}\Big), \qquad T_{\text{supply}} = T_{\text{amb}} - \frac{\dot{Q}_{\text{actual,cool}}}{\dot{m} c_p}. \tag{116}$$

If a coefficient of performance is supplied,

$$P_{\text{spot}} = \frac{\dot{Q}_{\text{actual,cool}}}{\text{COP}}. \tag{117}$$

The aeration-network inlet is then reset to the actual spot-cooler supply state,

$$T_{\text{air,in}} = T_{\text{supply,actual}}, \tag{118}$$

while the electrical heater is disabled:

$$P_{\text{tot}} = 0, \qquad I_{\text{tot}} = 0. \tag{119}$$

The post-cooler condensate calculation uses the dry-air flow rate $\dot{m}_{\mathrm{dry}}$ and the saturated outlet humidity ratio $w_{\mathrm{sat,out}}$:

$$\dot{m}_{\mathrm{cond,psych}} = \max[\dot{m}_{\mathrm{dry}}\,(w_{\mathrm{in}} - w_{\mathrm{sat,out}})\,,\,0]\,. \tag{120}$$

If a rated condensate capacity $\dot{m}_{\mathrm{cond,rated}}$ is provided, the applied condensate rate is

$$\dot{m}_{\mathrm{cond}} = \min(\dot{m}_{\mathrm{cond,psych}}, \dot{m}_{\mathrm{cond,rated}})\,, \tag{121}$$

otherwise $\dot{m}_{\mathrm{cond}} = \dot{m}_{\mathrm{cond,psych}}$. The supply humidity ratio is then

$$w_{\mathrm{out}} = \max\left(w_{\mathrm{in}} - \frac{\dot{m}_{\mathrm{cond}}}{\dot{m}_{\mathrm{dry}}},\,0\right). \tag{122}$$

The assist blower is represented by a linear static curve. With shutoff pressure $\Delta p_{\mathrm{shutoff}}$, rated-duty point $(\dot{V}_{\mathrm{rated}}, \Delta p_{\mathrm{rated}})$, and implied free-air flow $\dot{V}_{\mathrm{free}}$,

$$\dot{V}_{\mathrm{free}} = \dot{V}_{\mathrm{rated}} \frac{\Delta p_{\mathrm{shutoff}}}{\Delta p_{\mathrm{shutoff}} - \Delta p_{\mathrm{rated}}}, \tag{123}$$

$$\Delta p_{\mathrm{avail}}(\dot{V}) = \max\left[\Delta p_{\mathrm{shutoff}}\left(1 - \frac{\dot{V}}{\dot{V}_{\mathrm{free}}}\right),\,0\right]. \tag{124}$$

The total summer water-loss rate is

$$\dot{m}_{\mathrm{water,day}} = 86400\,N_{\mathrm{tubes}} \sum_i \dot{m}_{\mathrm{evap},i}, \tag{125}$$

the reported net cooling is

$$Q_{\mathrm{cool}} = \max(-Q_{\mathrm{bed,total}},\,0)\,, \tag{126}$$

and the corresponding lumped cooling requirement at the cooling setpoint is

$$Q_{\mathrm{req,cool}} = UA_{\mathrm{tot}}\,(T_{\infty,\mathrm{summer}} - T_{\mathrm{set,cool}})\,. \tag{127}$$

Type. Repository-specific surrogate model. Equations (**??**)–(**??**) are simplified sensible-capacity relations; (**??**)–(**??**) are psychrometric mass-balance relations; (**??**)–(**??**) are linearized blower approximations.
Repository-native narrative. `HeatTransferStudy/VermiHeatLoss.tex:L2084-L2284`.
Implementation. `heat_transfer_study.py:L2534-L2565`; `heat_transfer_study.py:L2605-L2705`.
Deviation note. The blower curve is a two-point linear approximation, not a full manufacturer fan map, and the spot-cooler model does not include a detailed refrigeration-cycle calculation.

15 Constraints and feasibility conditions

The current heating hard-constraint residual vector is

$$\mathbf{g}_{\text{heat}} = \begin{bmatrix} \max(0, I - I_{\max}) \\ \max(0, T_{\text{wire}} - T_{\text{wire,max}}) \\ \max(0, T_{\text{air,out}} - T_{\text{air,out,max}}) \\ \max(0, \Delta p - \Delta p_{\max}) \\ \max(0, \dot{m}_{\text{water}} - \dot{m}_{\text{water,max}}) \\ \max(0, u_h - u_{h,\max}) \\ \max(0, T_{b,\min} - T_b) \\ \max(0, T_{t,\min} - T_t) \\ \max(0, T_b - T_{b,\max}) \\ \max(0, T_t - T_{t,\max}) \\ \max(0, |T_b - T_t| - \Delta T_{\max}) \\ \max(0, \text{dutyNeeded} - 1) \\ \max(0, -Q_{\text{bed,total}}) \end{bmatrix}. \tag{128}$$

The current cooling hard-constraint residual vector is

$$\mathbf{g}_{\text{cool}} = \begin{bmatrix} \max(0, \Delta p - \Delta p_{\text{avail}}) \\ \max(0, \Delta p - \Delta p_{\max}) \\ \max(0, \dot{m}_{\text{water}} - \dot{m}_{\text{water,max}}) \\ \max(0, u_h - u_{h,\max}) \\ \max(0, T_{b,\min} - T_b) \\ \max(0, T_{t,\min} - T_t) \\ \max(0, T_b - T_{b,\max}) \\ \max(0, T_t - T_{t,\max}) \\ \max(0, |T_b - T_t| - \Delta T_{\max}) \\ \max(0, -Q_{\text{cool}}) \end{bmatrix}, \tag{129}$$

where $Q_{\text{cool}} = \max(-Q_{\text{bed,total}}, 0)$.

Type. Repository-defined feasibility residuals.
Repository-native narrative. Heating constraints appear in `HeatTransferStudy/VermiHeatLoss.tex:L1614-L1738`. The historical summer lexicographic block at `HeatTransferStudy/VermiHeatLoss.tex:L2251-L2279` is intentionally *not* treated as governing because the live Python code now uses constrained NSGA-II for cooling.
Implementation. `heat_transfer_study.py:L2220-L2451`; `heat_transfer_study.py:L2963-L3147`.
Deviation note. The cooling NSGA-II constraints require positive cooling capacity, but they do *not* currently enforce $Q_{\text{cool}} \geq Q_{\text{req,cool}}$. Cooling shortfall is computed for diagnostics, not as a hard feasibility constraint.

16 Objective functions and optimization quantities

Heating now uses constrained NSGA-II on the discrete candidate grid with objective vector

$$\mathbf{f}_{\text{heat}} = \begin{bmatrix} P_{\text{tot}} \\ -\min(T_b, T_t) \\ |T_b - T_t| \end{bmatrix}. \tag{130}$$

The reported heating design is then selected from the feasible Pareto set by the report-priority tuple

$$\pi_{\text{heat,report}} = \begin{bmatrix} -\min(T_b, T_t) \\ P_{\text{tot}} \\ |T_b - T_t| \\ -\frac{1}{2}(T_b + T_t) \\ \max(0, \text{dutyNeeded} - 1) \end{bmatrix}. \tag{131}$$

Cooling now uses constrained NSGA-II as well. The default cooling objective vector is

$$\mathbf{f}_{\text{cool}} = \begin{bmatrix} \max(T_b, T_t) \\ \frac{1}{2}(T_b + T_t) \\ P_{\text{spot}} + P_{\text{blower}} \end{bmatrix}, \tag{132}$$

and the reported cooling design is selected from the feasible cooling Pareto set by

$$\pi_{\text{cool,report}} = \begin{bmatrix} \max(T_b, T_t) \\ \frac{1}{2}(T_b + T_t) \\ P_{\text{spot}} + P_{\text{blower}} \\ \dot{V}_{\text{tot}} \\ |T_b - T_t| \\ -Q_{\text{cool}} \end{bmatrix}. \tag{133}$$

If no feasible cooling Pareto point exists, the code falls back to a best-available diagnostic ranking using the configured ordered tuple

$$\pi_{\text{cool,fallback}} = \left[g_{\sigma(1)}, g_{\sigma(2)}, \ldots, g_{\sigma(m)}\right], \tag{134}$$

where σ is the configured `priority_order` permutation over the implemented cooling metrics and residuals `heat_transfer_study.py:L3133-L3149`; `heat_transfer_study.py:L3173-L3177`; `heat_transfer_study.py:L3244-L3246`. This is a live fallback/reporting rule, not the stale historical summer primary-selection rule described in `HeatTransferStudy/VermiHeatLoss.tex:L2251-L2279`.

Type. Repository-defined optimization quantities.
Repository-native narrative. Heating NSGA-II is described in `HeatTransferStudy/VermiHeatLoss.tex:L1614-L1738`. The stale summer lexicographic primary-selection narrative in `HeatTransferStudy/VermiHeatLoss.tex:L2251-L2279` is intentionally excluded from the governing-equation set here because it is not implemented.
Implementation. `heat_transfer_study.py:L51-L121`; `heat_transfer_study.py:L2181-L2499`; `heat_transfer_study.py:L3148-L3208`; `heat_transfer_study.py:L6220-L6490`.
Deviation note. NSGA-II is used on an already evaluated discrete set. It is not replacing the physics solve with a continuous surrogate optimizer.

17 Numerical approximations, interpolation, and discretization

- The heating design space is generated by exhaustive discrete sweep in MATLAB, not by direct continuous optimization `vermicomposter_air_heater_design_model.m:L1172-L1476`.
- Electrical resistance depends on average wire temperature, so each operating point uses a short fixed-point iteration on $T_{\text{wire,avg}}$ `vermicomposter_air_heater_design_model.m:L1497-L1536`.

- Heater and aeration transport are solved by explicit axial marching over configured segment counts `vermicomposter_air_heater_design_model.m:L1803-L1999`; `heat_transfer_study.py:L2707-L2959`; `heat_transfer_study.py:L3775-L3959`.
- The summer branch solves for a bed-reference temperature by bracketing and bisection over the scalar residual relating branch cooling to the lumped required cooling target `heat_transfer_study.py:L2774-L2899`.
- Display interpolation in the plotting layer is presentation-only and does not alter the underlying evaluated candidate set `heat_transfer_study.py:L555-L562`.
- The annual climate layer uses periodic Gaussian-kernel regression over day-of-year and then overlays freeze-day and heat-wave events `heat_transfer_study.py:L4633-L4858`.
- Radial plume and habitat calculations use midpoint-grid union-area estimates rather than closed-form geometric intersection formulas `heat_transfer_study.py:L1383-L1595`.
- NSGA-II is applied to integer-valued indices or discrete point indices through `pymoo`; the optimizer explores the discrete repository candidate set rather than an interpolated continuous space `heat_transfer_study.py:L2410-L2499`; `heat_transfer_study.py:L3123-L3208`.

Detailed marching and fixed-point closures

The current heating solver uses the temperature-dependent wire resistivity

$$\rho_{\text{wire}}(T) = \rho_{\text{wire},20}\left[1 + \alpha_{\text{wire}}(T - 20^\circ\text{C})\right], \tag{135}$$

which yields a tube or branch resistance of the form already introduced in (**??**). The fixed-point iteration updates the average wire temperature by under-relaxation:

$$\overline{T}_{\text{wire}}^{(k+1)} = 0.65\,\overline{T}_{\text{wire}}^{(k)} + 0.35\,\overline{T}_{\text{wire,new}}^{(k)}, \tag{136}$$

and stops when

$$\left|\overline{T}_{\text{wire,new}}^{(k)} - \overline{T}_{\text{wire}}^{(k)}\right| < 0.05^\circ\text{C}, \tag{137}$$

or when the configured iteration cap is reached `vermicomposter_air_heater_design_model.m:L1497-L1536`. The winter aeration pressure-drop march uses the segment contribution

$$\Delta p_{\text{seg}} = \left(f\frac{\Delta x}{D} + \frac{K_{\text{minor}}}{N}\right)\frac{\rho u^2}{2}, \tag{138}$$

with the total candidate pressure drop assembled from the marched branch losses plus the distribution-network losses in (**??**) `vermicomposter_air_heater_design_model.m:L1953-L1956`; `vermicomposter_air_heater_design_model.m:L2123-L2232`.

The summer branch closes the bed-reference temperature by solving the scalar consistency condition

$$R(T_{\text{bed,ref}}) = \frac{T_b(T_{\text{bed,ref}}) + T_t(T_{\text{bed,ref}})}{2} - T_{\text{bed,ref}} = 0, \tag{139}$$

where T_b and T_t are obtained from the two-node bed solve using the branch heat rate predicted at that trial reference temperature `heat_transfer_study.py:L2910-L2917`. If the lower and upper trial bounds bracket a sign change, the code applies bisection:

$$T_{\text{mid}} = \frac{T_{\text{lo}} + T_{\text{hi}}}{2}, \tag{140}$$

updating the bracket by the sign of $R(T_{\mathrm{mid}})$ until either

$$|R(T_{\mathrm{mid}})| < 0.02 \qquad \text{or} \qquad |T_{\mathrm{hi}} - T_{\mathrm{lo}}| < 0.02, \tag{141}$$

otherwise the closer endpoint is retained `heat_transfer_study.py:L2918-L2947`. These are repository numerical constructs rather than textbook closed forms.

18 Annual climate and year-round operating layer

18.1 Climate anchors and regression baseline

The annual layer stores for each month m a Connecticut monthly mean temperature $\bar{T}_m$ and monthly standard deviation σ_m derived from the NOAA series described in `HeatTransferStudy/VermiHeatLoss.tex:L2576-L2616`. The representative-year ambient baseline uses circular Gaussian-kernel regression:

$$\hat{y}(d) = \frac{\sum_i w_i(d) y_i}{\sum_i w_i(d)}, \qquad w_i(d) = \exp\left[-\frac{1}{2}\left(\frac{\delta_i(d)}{h}\right)^2\right], \tag{142}$$

with wraparound day distance

$$\delta_i(d) = \min(|d - d_i|,\ P - |d - d_i|)\,. \tag{143}$$

This same periodic Gaussian regression is applied both to the monthly mean anchors and to the monthly standard-deviation anchors `heat_transfer_study.py:L4794-L4808`; `heat_transfer_study.py:L4866-L4879`.

18.2 Freeze-day allocation and anomaly override

Expected monthly freeze-day counts are converted to representative integer counts by largest-remainder allocation:

$$n_{f,m} = \mathrm{LR}(F_m). \tag{144}$$

Within each month, day-of-month weights are centered on the observed mean freeze day $\mu_{f,m}$:

$$w_{f,m}(i) = \exp\left[-\frac{(i - \mu_{f,m})^2}{2s_{f,m}^2}\right], \tag{145}$$

and sequential selection applies a repulsion modifier

$$\tilde{w}_{f,m}(i) = w_{f,m}(i) \prod_{j \in \mathcal{S}_{f,m}} \left[1 - 0.65 \exp\left(-\frac{(i-j)^2}{2r_{f,m}^2}\right)\right]. \tag{146}$$

On the selected freeze days, the ambient profile is overridden by

$$T_m(d) = \min\bigl(T_{\mathrm{base},m}(d),\, \bar{T}_m - \Delta T_{f,m}\bigr)\,. \tag{147}$$

These relations correspond to `heat_transfer_study.py:L4683-L4732`; `heat_transfer_study.py:L4893-L4906`.

18.3 Heat-wave allocation and anomaly override

Monthly spell starts and total heat-wave days are likewise allocated by largest-remainder rounding:

$$n_{s,m} = \mathrm{LR}(S_m), \qquad n_{h,m} = \mathrm{LR}(H_m). \tag{148}$$

The start-day weighting is

$$w_{h,m}(i) = \exp\left[-\frac{(i-\mu_{h,m})^2}{2s_{h,m}^2}\right], \tag{149}$$

with repulsion-modified score

$$\tilde{w}_{h,m}(i) = w_{h,m}(i) \prod_{j \in \mathcal{S}_{h,m}} \left[1 - 0.65 \exp\left(-\frac{(i-j)^2}{2r_{h,m}^2}\right)\right]. \tag{150}$$

Each selected heat-wave spell is forced to last at least three days in the code, and the ambient override is

$$T_m(d) = \max\big(T_m(d),\, \bar{T}_m + \Delta T_{h,m}\big)\,. \tag{151}$$

These relations correspond to `heat_transfer_study.py:L4735-L4791`; `heat_transfer_study.py:L4907-L4911`.

18.4 Daily heating and cooling loads

For each representative day d, the required heating and cooling loads are

$$Q_{\mathrm{req},h}(d) = UA_{\mathrm{bin}} \max(T_{h,\mathrm{set}} - T_\infty(d),\, 0)\,, \tag{152}$$

$$Q_{\mathrm{req},c}(d) = UA_{\mathrm{bin}} \max(T_\infty(d) - T_{c,\mathrm{set}},\, 0)\,, \tag{153}$$

which are implemented by rebuilding the bin model at the day's ambient and calling the same lumped-loss routine used elsewhere `heat_transfer_study.py:L4977-L4983`.

18.5 Reference-point capacity re-solve and duty application

The annual layer re-solves the selected heating and cooling operating points at each day's ambient temperature to obtain

$$Q_{h,\mathrm{cap}}(d) = Q_{\mathrm{to\,bed},h}(T_\infty(d))\,, \qquad P_{h,\mathrm{elec}}(d) = P_h(T_\infty(d))\,, \tag{154}$$

$$Q_{c,\mathrm{cap}}(d) = -Q_{\mathrm{to\,bed},c}(T_\infty(d))\,, \qquad P_{c,\mathrm{elec}}(d) = P_{\mathrm{spot\,cooler}}(d) + P_{\mathrm{assist\,blower}}(d). \tag{155}$$

The required duty fractions are

$$D_h^*(d) = \frac{Q_{\mathrm{req},h}(d)}{Q_{h,\mathrm{cap}}(d)}, \qquad D_c^*(d) = \frac{Q_{\mathrm{req},c}(d)}{Q_{c,\mathrm{cap}}(d)}, \tag{156}$$

and the applied duties are clipped at unity:

$$f_{\mathrm{day}} = \begin{cases} \min\left(\max\left(\dfrac{\dot{Q}_{\mathrm{load}}}{\dot{Q}_{\mathrm{nom}}}, 0\right), 1\right), & \dot{Q}_{\mathrm{nom}} > 0, \\ 0, & \dot{Q}_{\mathrm{nom}} = 0 \text{ and } \dot{Q}_{\mathrm{load}} \le 0, \\ 1, & \dot{Q}_{\mathrm{nom}} = 0 \text{ and } \dot{Q}_{\mathrm{load}} > 0, \end{cases} \tag{157}$$

which, in mode-specific notation, are exactly

$$D_h(d) = \min(\max(D_h^*(d), 0), 1), \qquad D_c(d) = \min(\max(D_c^*(d), 0), 1). \tag{158}$$

The applied bed load is therefore

$$Q_{\text{bed}}(d) = \begin{cases} D_h(d)\, Q_{h,\text{cap}}(d), & \text{heating day,} \\ -D_c(d)\, Q_{c,\text{cap}}(d), & \text{cooling day,} \\ 0, & \text{idle day.} \end{cases} \tag{159}$$

18.6 Daily bed temperatures, electricity, cost, and water

The daily bed temperatures are obtained from the same two-node steady system:

$$\begin{bmatrix} UA_{b,\infty} + UA_{bt} & -UA_{bt} \\ -UA_{bt} & UA_{t,\infty} + UA_{bt} \end{bmatrix} \begin{bmatrix} T_b(d) \\ T_t(d) \end{bmatrix} = \begin{bmatrix} f_b Q_{\text{bed}}(d) + UA_{b,\infty} T_\infty(d) \\ (1 - f_b) Q_{\text{bed}}(d) + UA_{t,\infty} T_\infty(d) \end{bmatrix}. \tag{160}$$

Daily heating and cooling electricity are

$$E_h(d) = \frac{P_{h,\text{elec}}(d)\, D_h(d)\, 24}{1000}, \qquad E_c(d) = \frac{P_{c,\text{elec}}(d)\, D_c(d)\, 24}{1000}, \tag{161}$$

so the total day electrical energy is

$$E_{\text{day}} = E_h(d) + E_c(d). \tag{162}$$

With electricity price c_{elec}, the daily cost is

$$C(d) = c_{\text{elec}} E_{\text{day}}, \tag{163}$$

and the daily water-replacement estimate is

$$m_{\text{water}}(d) = \dot{m}_{\text{water},24h}\, D(d), \tag{164}$$

where $D(d)$ is the active heating or cooling duty fraction for that day. These are repository numerical constructs driven by the same daily re-solved heating/cooling reference points `heat_transfer_study.py:L4966-L5072`.

19 Implementation notes and deviations from the written model

- The authoritative reported heating recommendation is now Python-side, even though MATLAB still generates the heating candidate physics. The rooted execution path therefore treats Python as the final recommendation authority `heat_transfer_study.py:L6908-L6947`; `vermicomposter_air_heater_design_model.m:L536-L608`.
- Maximum bottom and top bed temperatures are enforced in the Python-side heating and cooling feasibility logic; the MATLAB helper `explicitlySafe(...)` still omits those maxima `vermicomposter_air_heater_design_model.m:L2647-L2659`; `heat_transfer_study.py:L2220-L2368`; `heat_transfer_study.py:L2963-L3112`.
- Cooling selection is no longer lexicographic. The current code uses constrained NSGA-II plus a report-priority ordering inside the feasible cooling Pareto set `heat_transfer_study.py:L3148-L3208`; `heat_transfer_study.py:L6220-L6490`.

- Cooling shortfall is computed and printed, but not enforced as a hard cooling feasibility condition in the current NSGA-II implementation `heat_transfer_study.py:L2915-L2959`; `heat_transfer_study.py:L2963-L3112`.
- The post-cooler relative humidity used by the summer evaporation model is recomputed after psychrometric condensate removal, limited by any configured rated condensate capacity `heat_transfer_study.py:L2662-L2673`.
- The rooted refresh/export path disables MATLAB plotting and transients in the generated export helper, so the rooted execution graph exercises steady heating physics rather than the optional MATLAB transient branch `analyze_vermicomposter_heater.py:L294-L304`.

20 Symbol glossary with units

Symbol	Meaning	Units
A_h	area of one perforation hole	m^2
A_{wet}	effective wetted area assigned to evaporation	m^2
Bi	lumped-capacitance Biot number	1
c_p	specific heat of air	$\mathrm{J\,kg^{-1}\,K^{-1}}$
d_{32}	Sauter mean diameter used in particle-area estimate	m
D_i, D_o	inner and outer tube diameters	m
f	Darcy friction factor	1
h	convective heat-transfer coefficient	$\mathrm{W\,m^{-2}\,K^{-1}}$
h_m	mass-transfer coefficient	$\mathrm{m\,s^{-1}}$
h_{fg}	latent heat of vaporization	$\mathrm{J\,kg^{-1}}$
I_{tot}	total electrical current draw	A
Nu, Re, Pr, Ra	standard dimensionless groups	1
P_{tot}	total electrical power	W
$Q_{\mathrm{bed,total}}$	net total bed heat rate; negative in cooling mode	W
Q_{cool}	positive cooling capacity, $\max(-Q_{\mathrm{bed,total}}, 0)$	W
Q_{req}	lumped required heating or cooling load	W
T_b, T_t	bottom and top bed-node temperatures	°C
T_∞	ambient air temperature	°C
UA	lumped conductance	$\mathrm{W\,K^{-1}}$
u_h	mean hole exit velocity	$\mathrm{m\,s^{-1}}$
$\dot{m}_{\mathrm{evap}}$	evaporation mass flow rate	$\mathrm{kg\,s^{-1}}$
$\dot{V}_{\mathrm{tot}}$	total imposed volumetric flow rate	$\mathrm{m^3\,s^{-1}}$

21 Bibliography / source notes

R1 Churchill and Bernstein cylinder-in-crossflow citation string is stored in `vermicomposter_air_heater_design_model.m:L4-L8`. The repository preserves the authors, title, journal, year, and page range 300–306; no in-repository PDF was found for deeper page verification.

R2 Gnielinski's internal-flow correlation citation string is stored in `vermicomposter_air_heater_design_model.m:L9-L10`. Equation numbering was not verifiable from repository contents alone.

R3 Churchill friction-factor citation string is stored in `vermicomposter_air_heater_design_model.m:L11-L12`.

R4 Churchill–Chu natural-convection citation string is stored in `vermicomposter_air_heater_design_model.m:L13-L15`.

R5 The repository cites Bergman, Lavine, Incropera, and DeWitt generically for thermal resistances, developing internal flow, lumped balances, and free convection `vermicomposter_air_heater_design_model.m:L16-L17`. Edition and page numbers are not stored in the repository.

R7 The helix geometry relation is documented as a repository-native design relation in comments and TeX narrative `vermicomposter_air_heater_design_model.m:L1708-L1727`; `HeatTransferStudy/VermiHeatLoss.tex:L38-L121`.

R9 IAPWS saturation ancillary relations are cited in the repository commentary for water saturation properties `vermicomposter_air_heater_design_model.m:L18-L36`; `HeatTransferStudy/VermiHeatLoss.tex:L1089-L1167`.

R10 The Lewis relation is cited in the repository commentary for mass-transfer closure `HeatTransferStudy/VermiHeatLoss.tex:L1174-L1180`. No separate source PDF was found in the repository tree.

R11 Frederickson et al. Table 3 particle-size fractions are reproduced in the repository narrative `HeatTransferStudy/VermiHeatLoss.tex:L1015-L1088`. The repository does not include the original paper PDF.

- NOAA and EIA URLs used by the annual layer are preserved in the configuration and narrative, not in a dedicated bibliography database `HeatTransferStudy/VermiHeatLoss.tex:L2574-L3164`; `HeatTransferStudy/study_config.json:L289-L419`.

Implementation topic map

Concept	Governing equations and source status	Implementation mapping and notes
Tube flow, perforations, electrical grouping	Equations: `eq:volflow_per_tube`–`eq:electrical_totals`. Source status: repository derivation.	Implementation: `vermicomposter_air_heater_design_model.m:L763-L875`; `vermicomposter_air_heater_design_model.m:L1497-L1677`. Notes: series-parallel grouping allowed.
Internal convection and friction	Equations: `eq:internal_laminar_nu`, `eq:internal_turbulent_nu`, `eq:churchill_friction_full`. Source status: [R2], [R3], [R5].	Implementation: `vermicomposter_air_heater_design_model.m:L2382-L2407`; `heat_transfer_study.py:L3755-L3774`. Notes: used in both heater and aeration branches.
Heater axial energy balance	Equations: `eq:heater_segment_air`, `eq:wire_delta_t`. Source status: [R1], [R5] plus repository discretization.	Implementation: `vermicomposter_air_heater_design_model.m:L1803-L1894`; `heat_transfer_study.py:L3775-L3895`. Notes: explicit marching.
Aeration wall/jet/latent coupling	Equations: `eq:aeration_q_wall`–`eq:aeration_air_march`. Source status: repository reduction of sensible plus latent paths.	Implementation: `vermicomposter_air_heater_design_model.m:L1895-L1999`; `heat_transfer_study.py:L2815-L2959`. Notes: same structure in heating and cooling.

Concept	Governing equations and source status	Implementation mapping and notes
Evaporation and latent heat	Equations: `eq:sauter_diameter`–`eq:evap_selected`. Source status: [R1], [R9], [R10], [R11].	Implementation: `vermicomposter_air_heater_design_model.m:L891-L937`; `vermicomposter_air_heater_design_model.m:L2000-L2122`; `heat_transfer_study.py:L2567-L2603`; `heat_transfer_study.py:L3607-L3708`. Notes: minimum of three limiting rates.
Bin loss model and two-node bed	Equations: `eq:topmode_open`–`eq:two_node_split`. Source status: [R4], [R5] plus repository vented-cover extension.	Implementation: `vermicomposter_air_heater_design_model.m:L971-L1171`; `heat_transfer_study.py:L4326-L4438`. Notes: lumped bed closure.
Cooling spot cooler and assist blower	Equations: cooling and condensate relations in Sections 12–15. Source status: repository surrogate model.	Implementation: `heat_transfer_study.py:L2534-L3292`. Notes: includes condensate-limited RH update.
Heating and cooling NSGA-II	Equations: `eq:heating_constraint_residuals`–`eq:cooling_report_selection`. Source status: repository implementation.	Implementation: `heat_transfer_study.py:L51-L121`; `heat_transfer_study.py:L2181-L2499`; `heat_transfer_study.py:L3148-L3208`; `heat_transfer_study.py:L6220-L6490`. Notes: cooling now also NSGA-II.
Annual climate and duty layer	Equations: `eq:kernel_baseline`, `eq:daily_duty`, `eq:daily_energy`. Source status: repository numerical construct plus NOAA/EIA inputs.	Implementation: `heat_transfer_study.py:L4633-L5012`. Notes: not a literature first-principles climate model.